Introduction to the Linux Course

23/11/2025

Topics

- **Linux history and Linux distros**
  Link: https://www.youtube.com/watch?v=ShcR4Zfc6Dw
- **Unix philosophy**
  Link: https://youtu.be/gojeTqXdBH0?si=kg-iP4KEFjLU-Tto
- **Kernel vs OS**
  Link: https://youtu.be/IvGdY6luTtU?si=nEcir9iqjmYMa1lo
- **Free Software Philosophy**
  Link: https://youtu.be/Ag1AKIl$_2$$GM?si = 0YDnS2BDSrAAmFX8$
- **Why programmers love Linux**
  Link: https://youtu.be/otDOHt_Jges?si=4ChY6-xpYbOdbKQ6
- **Pros and cons**
  Link: https://youtu.be/pYfzZRyRzvs?si=4Q5_H0gJ3JHBodOc

Introduction to Linux

Welcome to the first step of your Linux journey. Whether you're a developer, cybersecurity student, or someone who just wants to understand how computers *really* work, this course will give you the foundation you need to actually **use** Linux—not just read about it.

This introduction will help you understand:

- What Linux actually is
- Why it became such a big deal

- How it's different from other operating systems
- Why developers, hackers, and engineers prefer it
- What mindset you should adopt while learning Linux
- How we'll approach the workshop

If this is your first time touching Linux: **perfect**. This course is made for you.

---

1. Why Linux Matters Today

If you look at the modern world—servers, cloud platforms, supercomputers, phones, routers, IoT, cybersecurity labs, programming environments—Linux is *everywhere*. It's the invisible engine running most of the technologies you interact with daily.

A few facts:

- **100%** of the world's top 500 supercomputers run Linux.
- **Most web servers** use Linux (Apache, Nginx).
- **Android**, the most widely-used mobile OS, is built on the Linux kernel.
- **Cybersecurity distributions** (Kali, Parrot) are Linux-based.
- Developers choose Linux for transparency, speed, customizability, and control.

So even if you've never installed Linux on your laptop, you've definitely used it indirectly.

---

2. Understanding Linux: What It Really Is

One of the biggest misconceptions is thinking Linux is "that thing that replaces Windows." Linux is not a program. It's not a GUI. It's not even an operating system by itself.

Linux = the kernel

The kernel is the core program that:

- Talks to hardware
- Manages memory

- Schedules processes
- Handles permissions
- Controls I/O

Everything else—the desktop, apps, package manager, and utilities—comes from the broader free-software ecosystem called **GNU**.

Together: **GNU tools + Linux kernel = GNU/Linux (a complete OS).**

This separation is important because Linux teaches you something deeper than using an OS—it teaches you *how systems work*.

---

3. Quick Overview of the UNIX Philosophy

Linux inherits its design principles from Unix. This philosophy will shape how you think during the whole workshop:

The UNIX Philosophy

1. **Write programs that do one thing, and do it well.**
2. **Write programs to work together.**
3. **Write programs to handle text, because text is a universal interface.**

This leads to:

- Tools that are small and powerful
- Commands that do simple tasks
- The magic happens when you combine them using pipes (‘|‘)

It’s why a Linux power-user is unstoppable—every small tool becomes a Lego block, and you can build anything from those blocks.

---

4. Linux History (The Simple Version)

Let’s keep it practical.

1969 – Unix born

ATT created Unix. Clean, elegant, powerful.

1983 – GNU Project launched

Richard Stallman began building a free Unix-like system. GNU had everything—compilers, libraries, tools—*except a kernel.*

1991 – Linus Torvalds writes the Linux Kernel

As a personal hobby project. He released it under a free license, allowing anyone to study, modify, and contribute.

1992 – GNU + Linux merged

This created the first fully free operating system.

1990s–2000s – Linux takes over servers

Why? Stability, control, security, no licensing fees.

2010s–2020s – Linux becomes everywhere

Cloud, containers, mobile, embedded systems, DevOps… The world quietly standardized on Linux.

That's why learning Linux today is a **career skill**, not just a personal preference.

---

5. Linux Distributions (Distros)

If Linux is the engine, a **distro** is the full car.
Different distros = different combinations of:

- Package managers
- Desktop environments
- Default tools
- Philosophy
- Target audience

Beginner-friendly distros

- Ubuntu
- Linux Mint
- MX Linux

Power-user distros

- Arch Linux
- Gentoo
- Void Linux

Enterprise distros

- Red Hat Enterprise Linux
- SUSE Linux Enterprise

Security distros

- Kali Linux
- Parrot OS

> Throughout the workshop, we'll use a distro that is: Stable, Easy to install, Common across many environments (Usually Ubuntu or Mint—you'll get confirmation before the workshop.)

---

6. Kernel vs Operating System

This is where newcomers usually get confused.

Kernel

- The lowest-level component
- Manages memory, processes, scheduling
- Talks to CPU, RAM, disks
- Loads drivers
- Handles system calls

Operating System

- Kernel
- Plus userland tools (GNU utilities)
- Plus services
- Plus package managers
- Plus desktop environments
- Plus apps

Linux is the *kernel*. GNU/Linux is the *operating system*.

This distinction matters because in this course you will interact with the kernel indirectly using tools like:

- `top`, `ps`, `dmesg`
- `/proc` filesystem
- System calls through C programs
- Permissions and user management

This duality is key to becoming a real system administrator or developer.

---

7. The Free Software Philosophy (Why Linux Exists)

Linux exists because of the free software movement. Free software is about *freedom*, not price.

The Four Essential Freedoms

1. **Freedom 0:** Run the program for any purpose
2. **Freedom 1:** Study the source code and modify it
3. **Freedom 2:** Distribute copies
4. **Freedom 3:** Distribute modified versions

If a program denies any of these freedoms, it becomes **proprietary**, which means:

- You can't see how it works

- You can't fix it
- You depend on the vendor
- You lose control over your own computer

Linux was built to solve this.
During the workshop, you'll see the impact of this philosophy:

- Software you can fully audit
- Tools you can modify
- Community-built contributions
- Transparency at every layer

This is why cybersecurity professionals prefer Linux—you can see *everything* happening in the system.

---

8. Why Programmers Love Linux

Software developers love Linux for reasons that will become obvious once you start using it:

1. Everything is a file

Devices, sockets, processes, IPC… all represented as files. Simplifies development massively.

2. Powerful terminal environment

You can automate anything.

3. Package managers

Install complete development stacks in seconds.

4. DevOps and cloud-native tools

Docker, Kubernetes, CI pipelines—all built on Linux.

5. Open-source ecosystem

Learn from real-world code. Fix things. Build tools.

6. High performance and stability

Perfect for servers and embedded systems.

7. Customizability

Modify your environment to match exactly how you think and work.

> This course will help you feel comfortable enough that Linux becomes your main development environment.

---

9. Pros and Cons of Linux

Pros

- Powerful CLI
- Free and open source
- Fast, lightweight
- Secure and stable
- Full control over your system
- Great for programming
- Great for networking
- Massive community support

Cons

- Some hardware drivers might be missing
- Gaming is improving but still imperfect
- Adobe apps not officially available
- Certain distros can be difficult to use
- Requires learning mindset, not point-and-click

But as a developer, engineer, or cybersecurity student, the *pros* outweigh the cons by a huge margin.

10. How to Think Like a Linux User

To succeed in this workshop:

Adopt these habits:

- **Be curious** — try commands, explore folders.
- **Read error messages** — Linux errors are usually helpful.
- **Use man pages** — every command has documentation.
- **Experiment safely** — break things in virtual machines.
- **Think in small tools** — pipes, filters, text processing.
- **Love the terminal** — it's your real power.

  Linux rewards people who explore.

11. What You'll Learn in This Course

This introduction sets the ground for the entire course. By the end of the workshop, you will:

System Skills

- Install and manage Linux
- Understand filesystems
- Work confidently in the terminal
- Manage users, processes, permissions
- Control systemd services
- Understand the kernel's role

Networking Skills

- Configure network interfaces
- Use diagnostic tools (ping, traceroute, curl)
- Manage firewall rules

Programming and Automation

- Shell scripting
- Using package managers
- Working with logs
- Troubleshooting

Security Awareness

- Permissions
- Privilege escalation basics
- Monitoring system behavior

You'll not just use Linux—you'll **understand** Linux.

Useful Resources

YouTube Channels

- anaHr — `https://www.youtube.com/@anaHr`
- mohammad besar — `https://www.youtube.com/@mmbesar`
- alwaqqad — `https://www.youtube.com/@alwaqad`
- sudostart — `https://www.youtube.com/@Sudo_Start`
- linuxtopia — `https://www.youtube.com/@linuxtopia`
- abdulmogeeb alhameed — `https://www.youtube.com/@abdulmogeeb`

Books

- **The Art Of UNIX Programming** — *Eric Raymond*

All rights reserved for TogetherTeam [NOT For Commercial Use] !!
contact the author via email khaledmhfz2004@gmail.com

Install Linux On Your Machine

25-11-2025

Topics

- **Choosing your distro**
  Link: https://youtu.be/8yVlJEzq2eg?si=SWcgh_Bw51Li3lv6
- **Install your Linux system**
  Link: https://www.youtube.com/watch?v=C-a5IamFIuM
- **Switch to Linux while keeping data from Windows / Dual-booting with Windows**
  Link: https://youtu.be/XevGfO_vQJQ?si=OBqaM8UlUeH1wjdQ
- **VirtualBox VM setup**
  Link: https://youtu.be/wX75Z-4MEoM?si=raCzHOzYJy-GZi9S
- **Post-install (drivers – firewall – backup and restore system)**
  Link: https://mmbesar.github.io/Tutorials/Ubuntu-24.04-Post/

1. Choosing Your Linux Distribution

Before installing Linux, you must pick the right **distro** (distribution). Think of distros like different flavors of the same operating system — they all share the Linux kernel but differ in user interface, package managers, performance, and philosophy.

1.1 What You Should Look For

When choosing a distro, consider:

Distro Selection Guide:

- **Ease of use** — Beginner-friendly UI, simple updates

  - Good Choices: Ubuntu, Linux Mint, Pop!_OS
- **Stability** — Fewer issues & long-term support
  - Good Choices: Ubuntu LTS, Debian
- **Performance** — Works well on older hardware
  - Good Choices: Xubuntu, Linux Lite
- **Security** — Good by default, strong community
  - Good Choices: Fedora, Ubuntu
- **Software availability** — Access to apps & drivers
  - Good Choices: Ubuntu, Fedora

1.2 Recommended Distro for Beginners

Ubuntu 24.04 LTS
Reason: stable, popular, lots of tutorials, works on almost all laptops.

1.3 Quick Tips

- If your PC is older than **10 years**, pick **Xubuntu** or **Linux Mint XFCE** or **MX-Linux** or **Zorin OS**.
- If you're into **gaming**, choose **Pop!_OS** or **Nobara OS**.
- If you want a macOS-like UI, choose **elementaryOS**.
- Avoid Arch-based systems as your first Linux unless you're ready for deeper learning.

---

2. Installing Your Linux System

In This section we will use Ubuntu 24.04 for Explanation.

2.1 Before You Install – VERY Important

A. Backup Your Files

The installer will delete the Windows partition if you're switching fully. Backup all files from:

- Desktop
- Documents
- Pictures
- Music
- Videos

Move them to:

- D:\(or another partition)
- External HDD
- USB stick

B. Check Your Boot Mode

Open Start → type **System Information** → look for:

- **UEFI** → modern, secure boot
- **Legacy BIOS** → older systems

This affects partitioning and bootloader behavior.

C. Download the ISO

- Ubuntu official website
- Size [5 - 6]GB

D. Create a Bootable USB

Recommended tool: **Ventoy**

- Supports multiple ISOs on the same flash drive
- Very beginner-friendly

2.2 Try Ubuntu Before Installing

Boot from the USB and choose:

Try Ubuntu (Live Mode)

You can test:

- Wi-Fi
- Bluetooth
- Sound
- Display
- Keyboard
- Touchpad

> If everything works → safe to install.

---

3. Installing Linux (Manual Partitioning)

This is the *critical* step — mistakes here delete data.

3.1 Choose Manual Installation

DO NOT choose:

- “Erase disk and install Ubuntu”
- “Install alongside Windows” (unpredictable for beginners)

Instead, choose: **Something Else (Manual Partitioning)**

3.2 Identify Windows Partitions to Delete

In the live session, open a terminal:

```
$ lsblk
```

You will see all disks:

- Windows system partitions
- Data partitions
- USB Ventoy

- Empty space

Delete only:

- The Windows partition (C drive)
- Windows system reserved partitions

Never delete your data partitions.

3.3 Creating Linux Partitions

Recommended layout:

Linux Partition Layout:

- **EFI/System** — 512–1024 MB — FAT32 — `/boot/efi` — Needed only for UEFI
- **ROOT** — 40–100 GB+ — EXT4 — `/` — Main system
- **SWAP** — Optional (2–4 GB) — swap — Only if RAM < 8GB
- **Data (Windows)** — Keep — NTFS — `/mnt/data1` — Do *not* format

Beginners Tip: Don't create a separate `/home` partition. You can add it later when you're more experienced.

---

4. Switching to Linux While Keeping Data (Dual Boot)

4.1 How Dual Boot Works

Your disk will contain:

- Windows NTFS partitions
- Linux EXT4 partitions
- ESP (bootloader)
- Shared data partitions

GRUB bootloader will let you choose the OS at startup.

4.2 Before Dual Booting

- Clean Windows (delete temp, uninstall unused programs)
- Run: `chkdsk /f`
- Disable Fast Startup in Windows
- Shrink Windows partition using *Disk Management*

4.3 Partition Strategy

You will create:

- EFI partition (use existing one)
- Root partition
- (Optional) Swap partition

Mount Windows data partitions as:

```
/mnt/data1
/mnt/data2
```

Never check the "format" box.

4.4 Accessing Windows Files After Installation

Open **Files → Other Locations → Ubuntu → mnt**.
Your data partitions appear there.
You can:

- Pin them
- Bookmark them
- Rename them

---

5. Installing Linux in VirtualBox (VM Setup)

For students who don't want to touch their main system.

5.1 Why a Virtual Machine?

- Practice Linux commands safely
- Test configurations
- Use Linux tools on a Windows/macOS system
- No risk of data loss

5.2 Basic Requirements

- CPU: Dual-Core
- RAM: 6–8 GB system → allocate 3–4 GB to VM
- Disk: 25–40 GB virtual disk
- GPU: Enable 3D Acceleration

5.3 Recommended VirtualBox Settings

- **Chipset:** ICH9
- **EFI:** Enabled for Ubuntu 24.04
- **CPU:** 2 cores
- **Display:** 128 MB VRAM
- **Network:** Bridged Adapter (for networking labs)

5.4 Installing Guest Additions

Inside VM → Devices → Insert Guest Additions CD
Run:

```
$ sudo apt install build-essential dkms linux-headers-$(uname -r)
$ sudo sh /media/username/VBox*/VBoxLinuxAdditions.run
```

This enables:

- Auto-resize display
- Clipboard sharing
- Drag & drop
- Better performance

6. Post-Installation Setup (Ubuntu 24.04)

6.1 Update System

```
$ sudo apt update && sudo apt upgrade -y
```

6.2 Install Ubuntu Restricted Extras

Adds codecs for:

- MP3
- MP4
- MOV
- Fonts

```
$ sudo apt install ubuntu-restricted-extras
```

6.3 Add Flatpak + Flathub

```
$ sudo apt install flatpak
$ flatpak remote-add --if-not-exists flathub
https://flathub.org/repo/flathub.flatpakrepo
```

Install Flatpak support in GNOME Software:

```
$ sudo apt install gnome-software gnome-software-plugin-flatpak
```

6.4 AppImage Support

```
$ sudo apt install libfuse2
```

6.5 GNOME Extensions & Tweaks

```
$ sudo apt install chrome-gnome-shell gnome-shell-extension-manager
gnome-tweaks
```

6.6 Minimize on Click

Classic:

```
$ gsettings set org.gnome.shell.extensions.dash-to-dock click-action
'minimize'
```

Newer behavior:

```
$ gsettings set org.gnome.shell.extensions.dash-to-dock click-action
'focus-minimize-or-appspread'
```

6.7 Enable New Document Menu

```
$ touch ~/Templates/new
```

6.8 Keyboard Layout Switching

Set:

```
$ gsettings set org.gnome.desktop.wm.keybindings switch-input-source
"['<Shift>Alt_L']"
$ gsettings set org.gnome.desktop.wm.keybindings
switch-input-source-backward "['<Alt>Shift_L']"
```

6.9 System Backups

Timeshift (System Snapshots)

```
$ sudo apt install timeshift
```

Déjà Dup (User files backup)

Built into Ubuntu.

6.10 Final System Update

```
$ sudo apt update && sudo apt upgrade -y && flatpak update -y && sudo
snap refresh
```

Conclusion

By the end of this session, students should be able to:

- ✓ Choose a Linux distro that fits their hardware and needs
- ✓ Install Linux safely and correctly
- ✓ Dual-boot without losing their Windows files
- ✓ Use Linux inside a Virtual Machine
- ✓ Apply essential post-install steps for performance, security, and usability

All rights reserved for TogetherTeam [NOT For Commercial Use] !!
contact the author via email khaledmhfz2004@gmail.com

Linux Command Line Basics

30/11/2025

1. Introduction to the Linux Command Line

The Linux command line (often called the **terminal**, **shell**, or **CLI**) is a text-based interface that allows you to interact with your system quickly and efficiently. While graphical interfaces (GUIs) are great for daily tasks, the command line provides:

- Faster automation
- Power-user flexibility
- Access to advanced tools
- A universal interface across all distros

> Most distributions ship with **bash** as the default shell, though others (zsh, fish) also exist. To open a terminal:

- **Ubuntu / Debian**: `Ctrl + Alt + T`
- **Fedora**: Activities → Terminal
- **Any distro**: Search "Terminal"

2. System Information

Linux provides several commands to check system details:

2.1 `uname`

Shows basic system information:

```
$ uname
$ uname -a
$ uname -r
$ uname -m
```

2.2 hostnamectl

Shows OS name, kernel, hardware:

```
$ hostnamectl
```

2.3 inxi

A more advanced tool (you may need to install it):

```
$ sudo apt update # update package index
$ sudo apt install inxi
$ inxi -F
```

Note: you need to update the package index before installing any package.

Displays CPU, GPU, RAM, kernel, drivers, etc.

2.4 lsb_release

Useful on Debian-based systems:

```
$ lsb_release -a
```

---

3. Shell Variables

The shell stores data (variables) temporarily during your session.

3.1 Viewing Variables

```
$ echo $HOME
$ echo $USER
$ echo $PATH
```

3.2 Creating Variables

```
$ NAME="Khaled"
$ echo $NAME
```

3.3 Exporting Variables

Makes them available to programs:

```
$ export EDITOR=nano
```

3.4 Listing All Variables

```
$ printenv
$ set
```

---

4. File System Navigation

Every Linux filesystem starts from the **root (/)** directory.

4.1 `pwd` – Print Working Directory

Shows where you currently are:

```
$ pwd
```

4.2 `ls` – List Files

```
$ ls
$ ls -l
$ ls -la
$ ls -h
```

4.3 `cd` – Change Directory

```
$ cd /home
$ cd Documents
$ cd ..
$ cd ~
$ cd -
```

Shortcuts:

- . → current directory
- .. → parent directory
- ~ → home directory
- - → previous directory

---

5. Essential File & Directory Commands

5.1 `mkdir` – Create Directory

```
$ mkdir projects
$ mkdir -p school/assignments
```

5.2 `touch` – Create Empty Files

```
$ touch notes.txt
```

5.3 cp – Copy

```
$ cp file.txt backup.txt
$ cp -r folder/ folder_backup/
```

5.4 mv – Move/Rename

```
$ mv oldname.txt newname.txt
$ mv file.txt /home/user/Documents/
```

5.5 rm – Remove

Dangerous but essential

```
$ rm file.txt
$ rm -r folder/
$ rm -rf folder/ # very dangerous
```

5.6 cat – View Content

```
$ cat file.txt
```

5.7 echo – Print Text

```
$ echo "Hello Linux!"
$ echo $USER
```

5.8 sleep – Pause Execution

```
$ sleep 5
```

---

6. Understanding Paths

6.1 Absolute Paths

Start from the root /.
Example:

```
/home/khaled/projects/code.py
```

6.2 Relative Paths

Start from your current location.
Example:

```
../images/photo.png
./run.sh
```

---

7. Package Management

Every Linux distribution uses a package manager to install software.

7.1 apt – Debian, Ubuntu, Linux Mint

Update repositories:

```
$ sudo apt update
```

Install:

```
$ sudo apt install gedit
```

Remove:

```
$ sudo apt remove gedit
```

Upgrade system:

```
$ sudo apt upgrade
```

7.2 dnf – Fedora, RHEL 8+, CentOS Stream

```
$ sudo dnf install nano
$ sudo dnf remove nano
$ sudo dnf update
```

7.3 yum – Older RHEL/CentOS

```
$ sudo yum install nano
$ sudo yum remove nano
$ sudo yum update
```

---

8. Editors

8.1 nano (Terminal Editor)

```
$ nano file.txt
```

Controls:

- Save: **Ctrl + S**
- Exit: **Ctrl + X**
- Search: **Ctrl + W**

8.2 gedit (GUI Editor)

```
$ gedit notes.txt &
```

The & lets the terminal continue running.

9. Getting Help

Linux has multiple built-in help resources.

9.1 `man` – Manual Pages

```
$ man ls
$ man tar
```

9.2 `info`

```
$ info coreutils
```

9.3 `tldr` – Simplified Examples

Install:

```
$ sudo apt install tldr
```

Use:

```
$ tldr ls
```

9.4 `--help`

```
$ ls --help
$ tar --help
```

9.5 `apropos` – Search for Commands by Description

```
$ apropos network
```

10. Archiving & Compression

10.1 tar (Most Common in Linux)

Create:

```
$ tar -cvf archive.tar folder/
```

Extract:

```
$ tar -xvf archive.tar
```

Compressed with gzip:

```
$ tar -czvf archive.tar.gz folder/
$ tar -xzvf archive.tar.gz
```

10.2 zip

```
$ zip archive.zip file1 file2
$ unzip archive.zip
```

10.3 7z (7-Zip)

Install:

```
$ sudo apt install p7zip-full
```

Compress:

```
$ 7z a archive.7z folder/
```

Extract:

```
$ 7z x archive.7z
```

11. Bonus Useful Commands

11.1 `clear` – clears the terminal

11.2 `history` – shows previous commands

11.3 `head` / `tail` – preview files

```
$ head file.txt
$ tail file.txt
$ tail -f log.txt
```

11.4 `less` – view files with navigation

```
$ less file.txt
```

11.5 `grep` – search text

```
$ grep "error" log.txt
```

11.6 `whoami` – current user

11.7 `df -h` – disk usage

11.8 `du -sh folder/` – folder size

---

12. Final Tips for Newcomers

- Practice every day
- Use `tldr` for quick examples
- Avoid `rm -rf` unless absolutely necessary
- Use tab completion
- Use `history` to track commands
- Explore `/etc`, `/home`, `/usr`, `/bin` directories carefully

Practice Exercises

1. Create a directory structure:

   ```
   projects/python/basics
   ```

2. Create three files inside `basics`.
3. Write your name into `info.txt` using `echo`.
4. Compress the folder using:
   - tar
   - zip
5. Install `tldr` and test 5 commands.
6. View your system kernel version and CPU info using two methods.

All rights reserved for TogetherTeam [NOT For Commercial Use] !!
contact the author via email khaledmhfz2004@gmail.com

Text Manipulation on Linux

09-12-2025

Topics

- **grep**
  Link: https://youtu.be/NO5sWPgj-44?si=RqN-cKCBSae4bvK7
- **sed**
  Link: https://youtu.be/EACe7aiGczw?si=q8Hyf9d1VgsO-uj2
- **awk**
  Link: https://youtu.be/9YOZmI-zWok?si=Larx5zKGgMMSrsn-
- **find**
  Link: https://youtu.be/BZ5gsFiIKOQ?si=WEDUXPu6DCi2anNL
- **Pipes and Redirections (> , » , | , <)**

🔗 Session 4 — Text Manipulation in Linux

Welcome to one of the most *useful and exciting* sessions in your Linux journey. In this session, we learn how Linux becomes a **superpower** when working with text, logs, automation, scripting, and filtering.

These commands are not just ”nice-to-have”. They are the *backbone* of every serious developer, sysadmin, DevOps engineer, hacker, or sleepy student debugging their broken assignment at 2:30 AM.

By the end of this guide, you will:

- ✔ Search like a pro using **grep**
- ✔ Transform data using **sed**
- ✔ Process structured text using **awk**

- ✔ Locate *anything* using **find**
- ✔ Combine commands like LEGO with **pipes & redirection**

And of course:

- ✔ Solve a bunch of hands-on exercises !!

Let's begin.

---

1. grep — The Search Engine of the Terminal

🔍 1.1 What is grep?

`grep` stands for **Global Regular Expression Print** — but don't worry, we don't need to understand 1970s developer humor to use it.

In simple words:

> **grep searches for text inside files or command output.**

🧠 1.2 Basic Syntax

```
$ grep [options] "pattern" file
```

Example:

```
$ grep "error" /var/log/syslog
```

Finds all lines containing *error*.

🎯 1.3 Useful Options

-i → ignore case

```
$ grep -i "linux" notes.txt
```

-r → recursive search in directories

```
$ grep -r "password" /etc
```

-n → show line numbers

```
$ grep -n "TODO" project.py
```

-v → invert match (show lines not matching)

```
$ grep -v "DEBUG" logs.txt
```

-E → extended regex

```
$ grep -E "cat|dog" animals.txt
```

1.4 Examples (real and practical)

Find all `.pdf` files listed in a directory:

```
$ ls | grep ".pdf"
```

Search inside all `.log` files:

```
$ grep "failed" *.log
```

Search for IP addresses:

```
$ grep -E "[0-9]+.[0-9]+.[0-9]+.[0-9]+" nginx.log
```

1.5 Exercises

1. **E1.** Find all lines containing the word `error` in `/var/log/syslog`.
2. **E2.** Use `grep` to find lines NOT containing "DEBUG".
3. **E3.** Recursively search for your name in your home directory.

> Take your time. Stretch. Drink water. Continue.

---

2. sed — The Stream Editor

2.1 What is sed?

`sed` is a **text transformer**.
You can use it to:

- ✔ Replace words
- ✔ Delete lines
- ✔ Insert text
- ✔ Extract text
- ✔ Edit files *without opening them*

2.2 Basic Syntax

```
$ sed 's/old/new/' file
```

This replaces the *first* occurrence of `old` with `new` per line.

Replace globally (all occurrences):

```
$ sed 's/old/new/g' file
```

2.3 Common sed operations

Replace text:

```
$ sed 's/linux/Linux/' notes.txt
```

Delete lines containing a word:

```
$ sed '/error/d' logs.txt
```

Print only lines matching a pattern:

```
$ sed -n '/INFO/p' logs.txt
```

Edit a file in-place:

⚠ BE CAREFUL (seriously)

```
$ sed -i 's/foo/bar/g' file.txt
```

2.4 Cool examples

Remove all spaces:

```
$ sed 's/ //g' text.txt
```

Replace multiple spaces with one:

```
$ sed 's/ */ /g'
```

Add prefix to each line:

```
$ sed 's/^/[INFO] /' file.txt
```

2.5 Exercises

1. **E5.** Replace all 2024 with 2025 in `report.txt`.
2. **E6.** Remove all lines containing the word "temp".
3. **E7.** Add the prefix `[LOG]` to each line of `output.txt`.
4. **E8.** Delete empty lines.

> You're doing great. Take a breath. Continue.

---

3. awk — The Text Processor King

3.1 What is awk?

`awk` is a **programming language**... hidden inside a terminal command.
It is perfect for:

- ✔ Extracting columns
- ✔ Summing numbers
- ✔ Processing CSV files
- ✔ Analyzing logs
- ✔ Building mini-scripts

3.2 Basic Syntax

```
$ awk 'pattern { action }' file
```

Example:

Display first column:

```
$ awk '{print $1}' file.txt
```

3.3 Field processing

Given a line:

```
Alice 23 Student
```

Fields are:

- `$1` = Alice
- `$2` = 23
- `$3` = Student

3.4 Useful tricks

Print only the second column:

```
$ awk '{print $2}' data.txt
```

Print lines where second field > 50:

```
$ awk '$2 > 50' data.txt
```

Sum column 3:

```
$ awk '{sum += $3} END {print sum}' sales.txt
```

3.5 Exercises

1. **E9.** Print only usernames from `/etc/passwd`.
2. **E10.** Sum the numbers in the second column of a file.
3. **E11.** Print rows where third column equals ”admin”.
4. **E12.** Print the first and last field of each line.

You are now officially becoming powerful.

4. find — The Ultimate Search Tool

🔍 4.1 What is find?

`find` searches your filesystem **recursively** for:

- ✔📄 Files
- ✔📁 Directories
- ✔⚖️ Sizes
- ✔🔒 Permissions
- ✔🔍 Names
- ✔📄 Types
- ✔ Everything

🧠 4.2 Basic Syntax

```
$ find path options
```

Example:

```
$ find . -name "*.txt"
```

🎯 4.3 Essential Options

Search by name:

```
$ find /home -name "notes.txt"
```

Case-insensitive:

```
$ find /home -iname "linux"
```

Search for directories:

```
$ find . -type d -name "src"
```

Search by size:

```
$ find . -size +1M
```

Search by permissions:

```
$ find / -perm 777
```

Exclude matching:

```
$ find . ! -name "*.txt"
```

4.4 Exercises

1. **E13.** Find all `.py` files in your home directory.
2. **E14.** Find all files larger than 10 MB.
3. **E15.** List all directories named `backup`.
4. **E16.** Find files with permission 777.

> Great! ☕ Grab some coffee. The final boss fight is next.

---

5. Pipes and Redirection

5.1 What are pipes?

Pipes (`|`) send the output of one command into another.

Think of it as plugging commands together like LEGO Pieces.

Example:

```
$ ls | grep ".txt"
```

→ 5.2 Output Redirection

Write to a file:

```
$ echo "Hello" > file.txt
```

Append to a file:

```
$ echo "World" >> file.txt
```

Redirect command output:

```
$ ls > list.txt
```

← 5.3 Input Redirection

```
$ command < file
```

Example:

```
$ wc -l < notes.txt
```

🔗 5.4 Combining pipes

Count python files:

```
$ ls | grep ".py" | wc -l
```

Get top 10 largest files:

```
$ du -ah | sort -h | tail
```

See unique words (sorted):

```
$ cat text.txt | tr ' ' \n' | sort | uniq
```

5.5 Exercises

1. **E17.** Count how many lines contain ”error” in `logs.txt`.
2. **E18.** Save all `.txt` filenames into `files.txt`.
3. **E19.** Print the first 5 unique words in `story.txt`.
4. **E20.** Display only lines containing ”user” from `/etc/passwd`.

Conclusion

Congratulations — you just learned the *core skills* that make Linux insanely powerful.
From now on:

- Logs won’t scare you
- Large files won’t slow you
- Searching becomes instant
- Processing data becomes easy
- Automation becomes natural
- Your terminal becomes a **superweapon**

This session prepares you for scripting, DevOps, automation, cybersecurity, and everything cool in Linux.

All rights reserved for TogetherTeam [NOT For Commercial Use] !!
contact the author via email khaledmhfz2004@gmail.com

User Permissions and File System on Linux

17-12-2025

Why This Topic Matters (Seriously)

Linux doesn't break because it's fragile. Linux breaks because **permissions exist** and **you don't understand them yet**.

Permissions:

- Protect your system
- Prevent malware from wrecking everything
- Make Linux powerful for servers, DevOps, security, and cloud

Once this clicks:

- Errors make sense
- "Permission denied" stops ruining your day
- You stop nuking your system with `sudo rm -rf`

1 Linux File System Hierarchy (The Big Picture)

Linux doesn't use drive letters like Windows (`C:\`, `D:\`). Instead, **everything starts at one place**:

/ — The Root of Everything

/ is the **top of the file system tree**.

Think of it like:

The trunk of a tree 🌲
Everything grows from here.

```
$ /
   home
   etc
   var
   usr
   bin
   sbin
   tmp
```

⚠ You **do not** casually mess around in `/`.

`/home` — Where Humans Live

This is where **users live**.

```
$ /home/chuck
$ /home/student
```

Inside your home directory:

- Documents
- Downloads
- Config files
- SSH keys
- Hidden dot files

Your home is:

- Owned by you
- Writable by you
- Mostly safe from system damage

👉 **Best Practice:**
If you're learning Linux, live in `/home`

`/etc` — Configuration Central

This is one of the **most important directories**.
`/etc` contains:

- System configuration files
- User settings
- Service configs
- Network configs

Examples:

```
$ /etc/passwd
$ /etc/shadow
$ /etc/hosts
$ /etc/sudoers
```

⚠ Editing files here can:

- Break login
- Break networking
- Lock you out completely

👉 **Rule:**
`/etc` = configuration, not programs

`/var` — Variable Data (Stuff That Changes)

This is where Linux stores:

- Logs
- Caches
- Spools
- Databases

Examples:

```
$ /var/log
$ /var/log/syslog
$ /var/log/auth.log
```

👉 If a system is acting weird:
👉 **Try Checking `/var/log` or Any other related logs for your problem**

`/usr` — User Programs (Not Users)

Confusing name. This is **not user home directories**.
`/usr` contains:

- Installed programs
- Libraries
- Shared resources

```
$ /usr/bin
$ /usr/lib
$ /usr/share
```

Most software you install ends up here.

---

2 📄 "Everything Is a File" (Linux Philosophy)

This is where Linux becomes *cool*.
In Linux:

- Files
- Directories
- Devices
- Hardware
- Network interfaces

👉 **Everything is a file**
Examples:

```
$ /dev/sda # hard drive
$ /dev/null # black hole
$ /proc/cpuinfo # CPU info
$ /dev/tty # current terminal
$ /dev/random # random number generator
$ /dev/urandom # faster random number generator
$ /proc/uptime # system uptime
$ /proc/net/tcp # TCP connections
$ /sys/class/net/eth0/address # network MAC address
$ /sys/class/backlight/brightness # screen brightness
$ /sys/class/power_supply/BAT0/capacity # battery level
$ /dev/loop0 # loopback device (mount files as disks)
```

Want hardware info?

```
$ cat /proc/cpuinfo
```

Want to silence output?

```
$ command > /dev/null
```

👉 This design is WHY:

- Linux is scriptable
- Linux is automatable
- Linux runs the internet

---

3 Dot Files (Hidden Power)

Files that start with a dot `.` are **hidden**.
Example:

```
$ .bashrc
$ .profile
$ .gitconfig
$ .ssh/
```

They usually store:

- Shell configs

- App preferences
- User settings

List them:

```
$ ls -a
```

Why hide them?

- Prevent clutter
- Prevent accidental deletion

> 👉 **Important:**
> Deleting dot files can reset your environment or break tools.

---

4 👤 File Ownership (Who Owns What?)

Every file has:

- **Owner**
- **Group**
- **Permissions**

Check with:

```
$ ls -l
```

Example:

```
$ -rwxr-xr-- 1 ahmad staff script.sh
```

Breakdown:

```
Owner: ahmad
Group: staff
```

Ownership matters because:

- Owners control permissions
- Groups allow shared access
- Others are everyone else

Changing Ownership

```
$ chown user file
$ chown user:group file
```

Example:

```
$ sudo chown root:root config.conf
```

⚠ Wrong ownership = broken apps

---

5 🔒 Permissions (rwx) — The Core Skill

Permissions are **everything**.
Three types:

- **r** = read
- **w** = write
- **x** = execute

Three levels:

- Owner
- Group
- Others

Example:

```
$ -rwxr-xr--
```

Break it down:

```
Owner: rwx
Group: r-x
Others: r--
```

Permission Numbers (Octal Mode)

This is where beginners panic—but don't.

Values:

```
r = 4
w = 2
x = 1
```

Add them:

```
7 = rwx
6 = rw-
5 = r-x
4 = r--
```

Example:

```
$ chmod 755 script.sh
```

Meaning:

```
Owner: 7 (rwx)
Group: 5 (r-x)
Others: 5 (r-x)
```

👉 You will see 755 and 644 everywhere.

Making a Script Executable

```
$ chmod +x script.sh
```

Without execute permission:

```
$ ./script.sh
# Permission denied
```

---

6 Root User (Unlimited Power)

Root is:

- The system administrator
- The power mode user
- The ”I can change everything” account

Root can:

- Read any file
- Delete any file
- Change any permission

`sudo` — Temporary Root

```
$ sudo command
```

This:

- Executes command as root
- Logs actions
- Requires password

> 👉 **Best practice:**
> Use `sudo`, not permanent root login

`su` — Switch User

```
$ su
$ su username
```

This:

- Switches shell to another user
- Requires target user password

> ⚠ Dangerous if misused

`/etc/sudoers` — Who Can Use sudo

DO NOT edit directly.
Always use:

```
$ sudo visudo
```

Example entry:

```
ahmad ALL=(ALL) ALL
```

⚠ Wrong syntax here => no sudo => bad day.

---

7 🔗 Hard Links vs Soft Links (Symbolic Links)

Links are **references** to files.

Hard Links

- Point to the same inode
- File exists in multiple locations
- Deleting one doesn't remove data

```
$ ln file1 file2
```

Limitations:

- Same filesystem only
- Can't link directories (usually)

Soft Links (Symlinks)

- Pointer to file path
- Like Windows shortcuts

```
$ ln -s target linkname
```

Example:

```
$ ln -s /var/log/syslog syslog_link
```

If target is deleted:

⚠ Link breaks

👉 Most commonly used type.

---

8 ⚠ Common Beginner Mistakes (Read This Twice)

This section will save your system.

⚠ Running Everything with sudo

Bad:

```
$ sudo npm install
$ sudo python script.py
```

Why bad?

- Files owned by root
- Permission nightmares
- Security risk

👉 Use sudo **only when required**

⚠ `chmod 777` Everything

```
$ chmod 777 file
```

This means:

Everyone can do everything

Bad for:

- Security
- Best practices
- Your future job interviews ☻

⚠ Editing System Files Without Backup

Before:

```
$ sudo cp config.conf config.conf.bak
```

Always.

⚠ Deleting Random Stuff in `/etc` or `/usr`

If you don't know what it does: 👉 **Don't delete it**

⚠ Not Reading Error Messages

Linux tells you:

- What failed
- Why it failed
- What permission is missing

Read it.

---

9 👉 Real-World Permission Scenarios

- **"Permission denied" when running script**
  - Possible Missing execute bit
- **App won't start**
  - Maybe Wrong ownership in config directory
- **Can't write to directory**
  - Check group permissions
- **Service won't start**
  - Maybe Config owned by wrong user

10 Final Linux Mindset Shift

Linux is not hard.
Linux is:

- Honest
- Strict
- Predictable

Permissions are not obstacles. They are **guardrails**.
Once you understand:

- File hierarchy
- Ownership
- Permissions
- Root access

You unlock:

- Servers
- DevOps
- Security
- Cloud
- Automation

What You Should Practice Next

- ✔ `ls -l` everywhere
- ✔ `chmod` with numbers
- ✔ `chown` safely
- ✔ Explore `/etc` (read-only)
- ✔ Break things **inside a VM**

If you want, next we can:

- Do a **hands-on lab**
- Build a **permission challenge**
- Simulate **real server mistakes**
- Or connect this to **Docker & servers**

Now go grab that coffee ☕
You just leveled up in Linux. 🐧🔥

All rights reserved for TogetherTeam [NOT For Commercial Use] !!
contact the author via email khaledmhfz2004@gmail.com